CSCI 3060U Software Quality Assurance, Fall 2026

Course Project Phase 2: Front End Rapid Prototype

Front End Design Document

Digital Distribution Service for Games: the point-of-purchase Front End (C++)

Group 15 member	Student ID	GitHub account
Raihan Mosharrof	100823087	Raihan-Mosharrof
David Zajac	100965471	DavidZajac123
Sanjith Krishnamoorthy	100877316	sqnjay

Due: Monday, October 19th, 2026, 8:00pm (Canvas, plus all files committed to GitHub)

Repository: https://github.com/DavidZajac123/Software-Quality-Assurance-Group-15.git

Contents

1. Purpose and scope
2. Inputs and outputs (Figure 1)
3. Architecture overview (Figure 2)
4. Detailed class design (Figures 3, 4 and 5)
5. Class and method intention table
6. Run-time behaviour (Figure 6)
7. Daily Transaction File records
8. Requirements coverage and Phase 2 scope

1. Purpose and scope

This document gives the design of the Front End of the point-of-purchase console program. It shows the overall structure of the solution, every class and method in human-readable UML class diagrams (Sections 3 and 4), and the intention of each class and method in one sentence (Section 5, Table 4). Sections 2 and 6 show where every input and output fits and how a transaction runs from start to finish.

Scope. The design covers every transaction in the project specification (login, logout, create, delete, sell, buy, refund and addcredit), the list_games transaction added by the client, the list_accounts transaction added in the client's Week 4 update, and the client clarifications and group decisions recorded in the Phase 1 Test Plan.

First version. The Phase 2 source code implements this design as the first version that runs on sample inputs. It is deliberately not yet tested against our Phase 1 requirements tests.

2. Inputs and outputs

The Front End is a console program: it reads standard input and three master files, and writes standard output, one Daily Transaction File (DTF) per session and, for start-up errors only, standard error. Figure 1 shows each one; Table 1 gives its format, when it is used, and the class that handles it. Apart from `main`'s start-up checks, only `Console`, `MasterFileReader` and `DailyTransactionWriter` read or write a stream or file.

Table 1. Inputs and outputs

Input or output	Format	When it is used	Handled by
Command line (in)	`frontend <users file> <games file> <collection file> <dtf prefix>`	Start-up	`main`
Standard input (in)	One transaction code or field value per line. A trailing carriage return is removed; a blank line is a value.	Throughout the run	`Console::readLine`, `Console::prompt`
Current User Accounts File (in)	28-character lines: username(15) type(2) credit(9), then an END line	At every login, before the username prompt	`MasterFileReader::readUserAccounts`
Available Games File (in)	49-character lines: game name(26) seller(15) price(6), then an END line	At every login, after the username is accepted	`MasterFileReader::readAvailableGames`
Game Collection File (in)	42-character lines: game name(26) owner(15), then an END line	At every login, after the username is accepted	`MasterFileReader::readGameCollection`
Standard output (out)	Each prompt and message on its own line; error messages start with "Error:"	Throughout the run	`Console::prompt`, `Console::showMessage`
Daily Transaction File (out)	File `<dtf prefix>_session<N>.txt`: one record per successful transaction in session order, then the 00 record (Section 7)	At every logout (session N of the run)	`DailyTransactionWriter::record...` methods, `writeSessionFile`
Standard error and exit status (out)	Start-up errors only (wrong number of arguments, or a master file that cannot be opened), then exit status 1; otherwise exit status 0 when the input ends	Start-up; end of run	`main`, `FrontEnd::run`

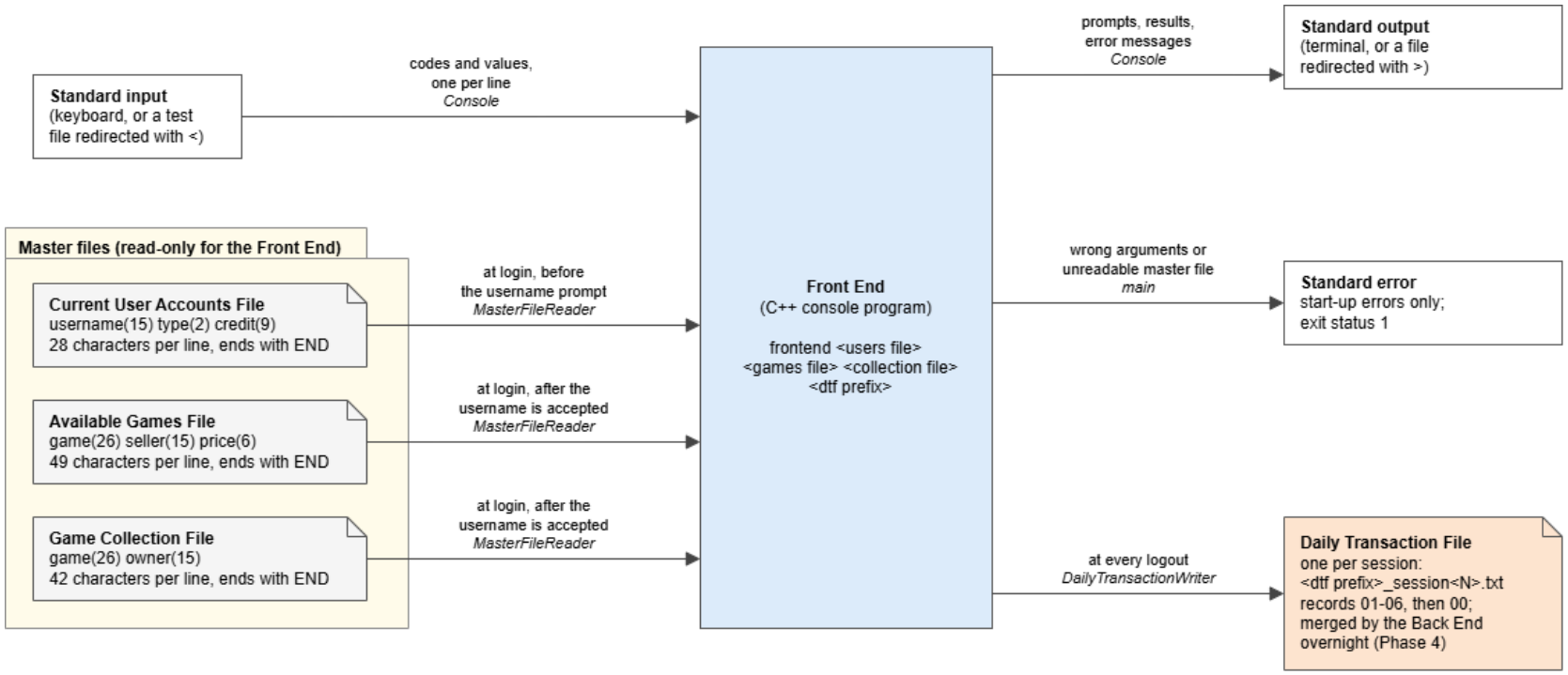


Figure 1. *Front End inputs and outputs. Arrows show the direction of data; the italic names are the classes that handle each input or output (standard error is used only by `main`, for start-up errors). The Back End later merges the session DTFs and writes the next day's master files.*

Example run. The command below reads the transaction stream from `buy_01_input.txt`, writes prompts and messages to `buy_01_output.txt`, and writes the session's DTF to `results/buy_01_session1.txt`. A second login in the same run would write `results/buy_01_session2.txt`.

```
frontend current_users.txt available_games.txt game_collection.txt results/buy_01 < buy_01_input.txt > buy_01_output.txt
```

Why three input files. The specification names two Front End input files. We also read the Game Collection File, because the buy rule "the buyer must not already have a copy of the game" cannot be checked without it.

3. Architecture overview

The classes are grouped into six packages (Table 2). Figure 2, on the next page, shows every class and how they relate; the members of each class follow in Section 4.

Size. The design has 24 classes, structs and enums plus the main program, and 92 methods (constructors and `main` included). Each method is small: each is designed to stay below a McCabe cyclomatic complexity of 10.

Table 2. Packages

Package	Classes	Responsibility
Program entry	main	Starts the program from the command line.
Control	FrontEnd	Runs the transaction loop and decides which transactions may run.
Transactions	Transaction (abstract), LoginTransaction, LogoutTransaction, CreateTransaction, DeleteTransaction, SellTransaction, BuyTransaction, RefundTransaction, AddCreditTransaction, ListGamesTransaction, ListAccountsTransaction	One class per transaction code; each owns its prompts, rules and DTF record.
Session state	Session, UserAccount, GameListing, OwnedGame, UserType	What the Front End knows during one login, and the records it is made of.
Input / Output	Console, MasterFileReader, DailyTransactionWriter	Every place data enters or leaves the program: console, master files, DTF files.
Utilities	UserTypeCodes, Money, FixedWidthFormat, InputRules	Shared conversions and limits, so no rule or format is written twice.

3.1 How a transaction flows through the design

1. `main` checks the four arguments and creates the `FrontEnd`, which owns one `Console`, `MasterFileReader`, `DailyTransactionWriter` and `Session`, and one object of each of the ten transaction classes.
2. `FrontEnd::run` reads one line at a time and passes it to `processTransactionCode`, which looks it up among the ten transaction codes.
3. `rejectionReason` applies three gates in order: nobody is logged in yet, someone is already logged in, or the account type is not permitted. A rejected code prints one error message, and no further line is read for it.
4. An accepted code calls the transaction's `execute(session, console, dtfWriter)`. `execute` prompts for all of the transaction's fields first, then calls `validateInput`.
5. If `validateInput` returns an error message, `execute` prints it and stops: nothing changes and no DTF record is added. Otherwise `execute` updates the `Session`, adds a record through the `DailyTransactionWriter`, and prints a confirmation.
6. `logout` writes the session's records and the 00 record to a new DTF and ends the `Session`. The loop then accepts only login, until the input ends.

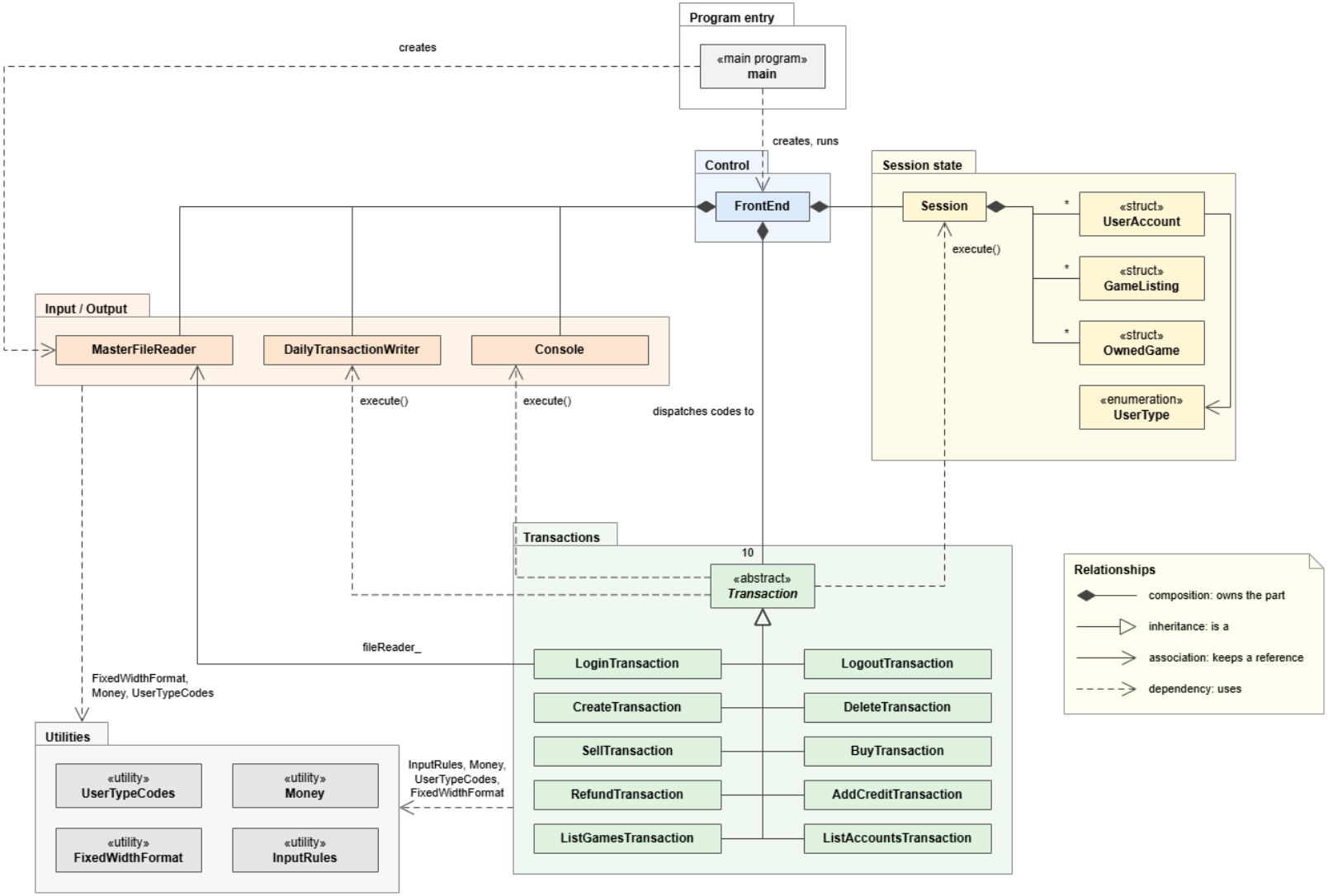


Figure 2. *Class diagram overview: every class, struct and enum, with members hidden. Filled diamonds are composition (the owner creates and holds the part); hollow triangles are inheritance; plain arrows are associations (a kept reference); dashed arrows are dependencies (every transaction's `execute()` uses `Session`, `Console` and `DailyTransactionWriter`; the transactions use all four utilities, `InputRules`, `Money`, `UserTypeCodes` and `FixedWidthFormat`; the file classes use `FixedWidthFormat`, `Money` and `UserTypeCodes`); "10" and "*" are multiplicities.*

3.2 Design principles

- **One class per transaction.** Each transaction code has its own subclass of the abstract `Transaction` (the Command pattern). `FrontEnd` treats all ten the same way through `code`, `requiresLogin`, `isPermittedFor` and `execute` (polymorphism), so changing a transaction touches one class.
- **Every rule and format in exactly one place.** Limits live in `InputRules`, field widths and padding in `FixedWidthFormat`, money conversion in `Money`, account-type codes in `UserTypeCodes`, and record layouts in `DailyTransactionWriter`. No code is cloned between transactions.
- **State behind one interface.** All in-session data lives in `Session` and changes only through its methods (encapsulation). Transactions never edit balances, accounts or listings directly.
- **Input and output isolated.** Apart from `main`'s start-up checks, only `Console`, `MasterFileReader` and `DailyTransactionWriter` read or write streams and files. The Back End depends only on the DTF format, which matters when Front Ends are swapped later in the course.
- **Simplest solution.** No global variables, no inheritance outside the transaction hierarchy, no configuration files, and no feature that is not in the requirements.

4. Detailed class design

Figures 3, 4 and 5 show every attribute and method. **Notation:** + public, - private, underlined = static, italic = abstract (implemented by every subclass). A parameter marked `out` is filled in by the method. `string` means `std::string`, passed by const reference. A * result may be `nullptr` (nothing found). `= value` after an attribute is its initial value. Every amount of money is a whole number of cents (`long long`).

4.1 Program entry, control and session state

- `main` only checks the command line and that the three master files open, then creates and runs `FrontEnd` (Section 2).
- `FrontEnd` owns every long-lived object. `transactions_` maps each code word (for example "buy") to its `Transaction` object.
- `Session` is rebuilt at every login from freshly read master files (`begin`) and closed at logout (`end`), so nothing carries over from one login to the next. During a session it is the only place where accounts, balances, listings and ownership change. Its two read-only views, `accounts()` and `listings()`, let list_accounts and list_games display that data without being able to change it.
- `UserAccount`, `GameListing` and `OwnedGame` are plain records: each holds one line of a master file.

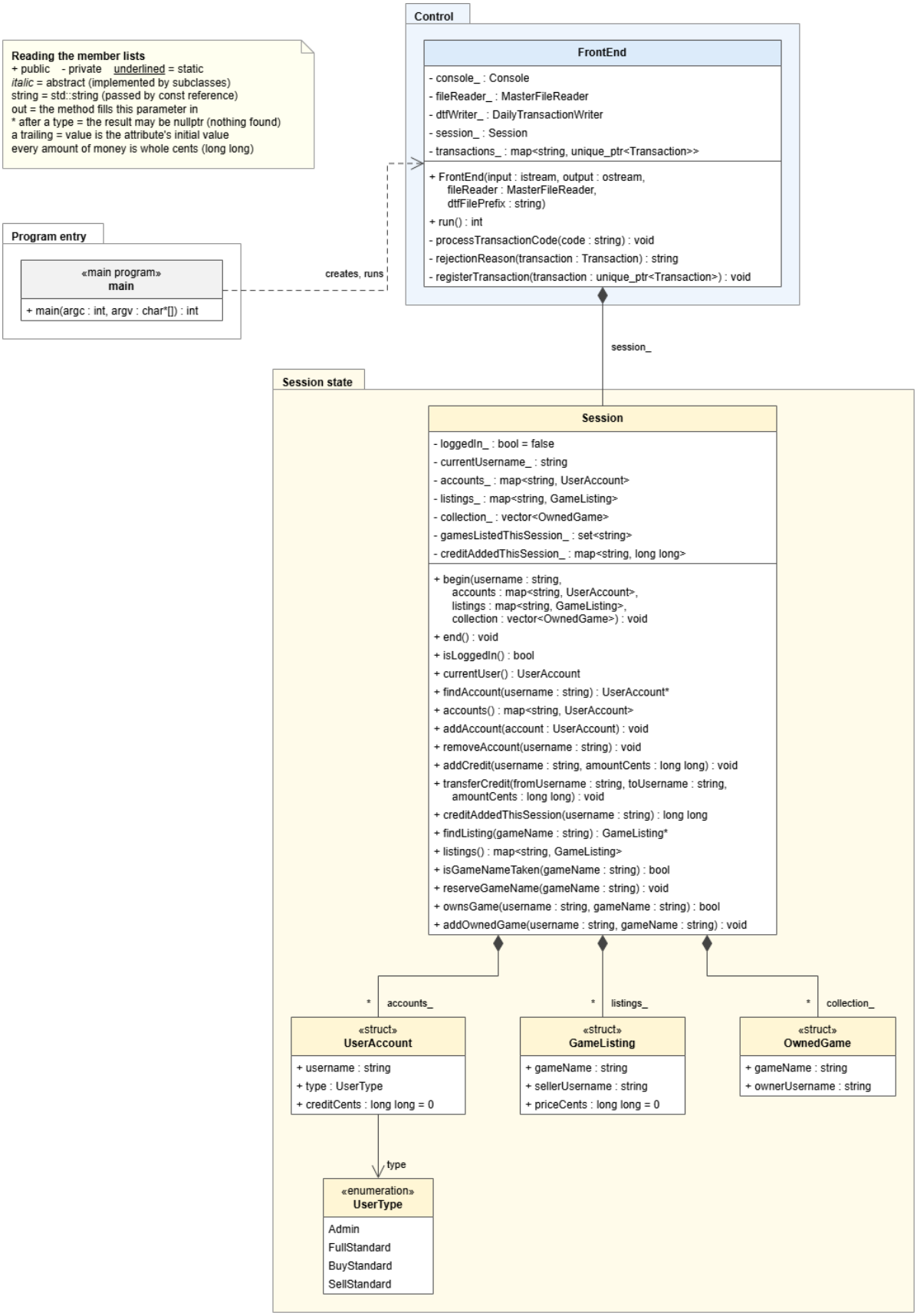


Figure 3. *Detailed classes: program entry, control and session state.*

4.2 Input/output and utilities

- `Console` removes a trailing carriage return from each line and returns false at the end of the input, which ends the transaction loop.
- `MasterFileReader` reads each master file up to its END line with `readRecordLines`, then parses the fixed-width fields with `FixedWidthFormat::extractField`, `Money::parse` and `UserTypeCodes::fromFileCode`. A line that cannot be parsed is skipped.
- `DailyTransactionWriter` keeps a session's records in memory until `writeSessionFile`; Section 7 gives each layout.
- The four utilities have only static members: constants (the field widths in `FixedWidthFormat`, the limits in `InputRules`) and static methods.

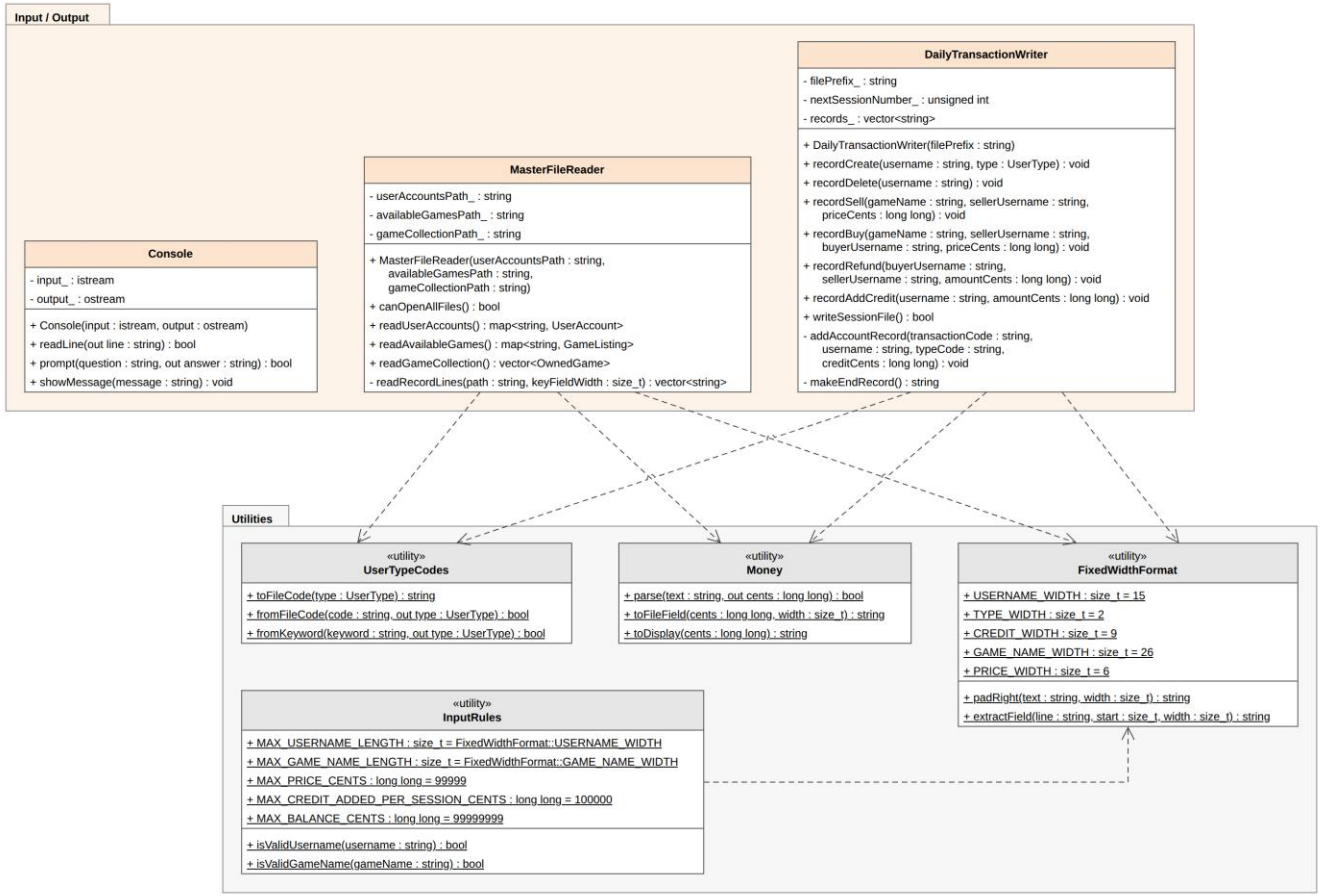


Figure 4. *Detailed classes: input/output and utilities. Dashed arrows are dependencies: the utilities each file class uses, and* `InputRules` *taking its two length limits from the* `FixedWidthFormat` *widths.*

4.3 Transactions

Every concrete transaction overrides `code` and `execute`; `CreateTransaction`, `DeleteTransaction`, `SellTransaction`, `BuyTransaction`, `RefundTransaction` and `ListAccountsTransaction` also override `isPermittedFor`, and `LoginTransaction` overrides `requiresLogin`. `ListAccountsTransaction`, added for the client's Week 4 update, only displays the session's accounts and writes no DTF record. Each transaction that checks two or more rules has a private `validateInput` that returns an error message, or an empty string when the input is valid; login's single check (does the account exist?) is made inside its `execute`. Table 3 summarises who may use each transaction, what it asks for, and what it records.

Table 3. Transactions at a glance

Class	Code	Accepted for	Prompts, in order	DTF record
`LoginTransaction`	login	Anyone, only while nobody is logged in	username	none
`LogoutTransaction`	logout	Any logged-in user	(none)	writes the DTF, ending with 00
`CreateTransaction`	create	Admin only	new username; account type	01
`DeleteTransaction`	delete	Admin only	username	02
`SellTransaction`	sell	Every type except buy-standard	game name; price	03
`BuyTransaction`	buy	Every type except sell-standard	game name; seller's username	04
`RefundTransaction`	refund	Admin only	buyer's username; seller's username; amount	05
`AddCreditTransaction`	addcredit	Any logged-in user	amount (admin: then username)	06
`ListGamesTransaction`	list_games	Any logged-in user	(none)	none (display only)
`ListAccountsTransaction`	list_accounts	Admin only	(none)	none (display only)

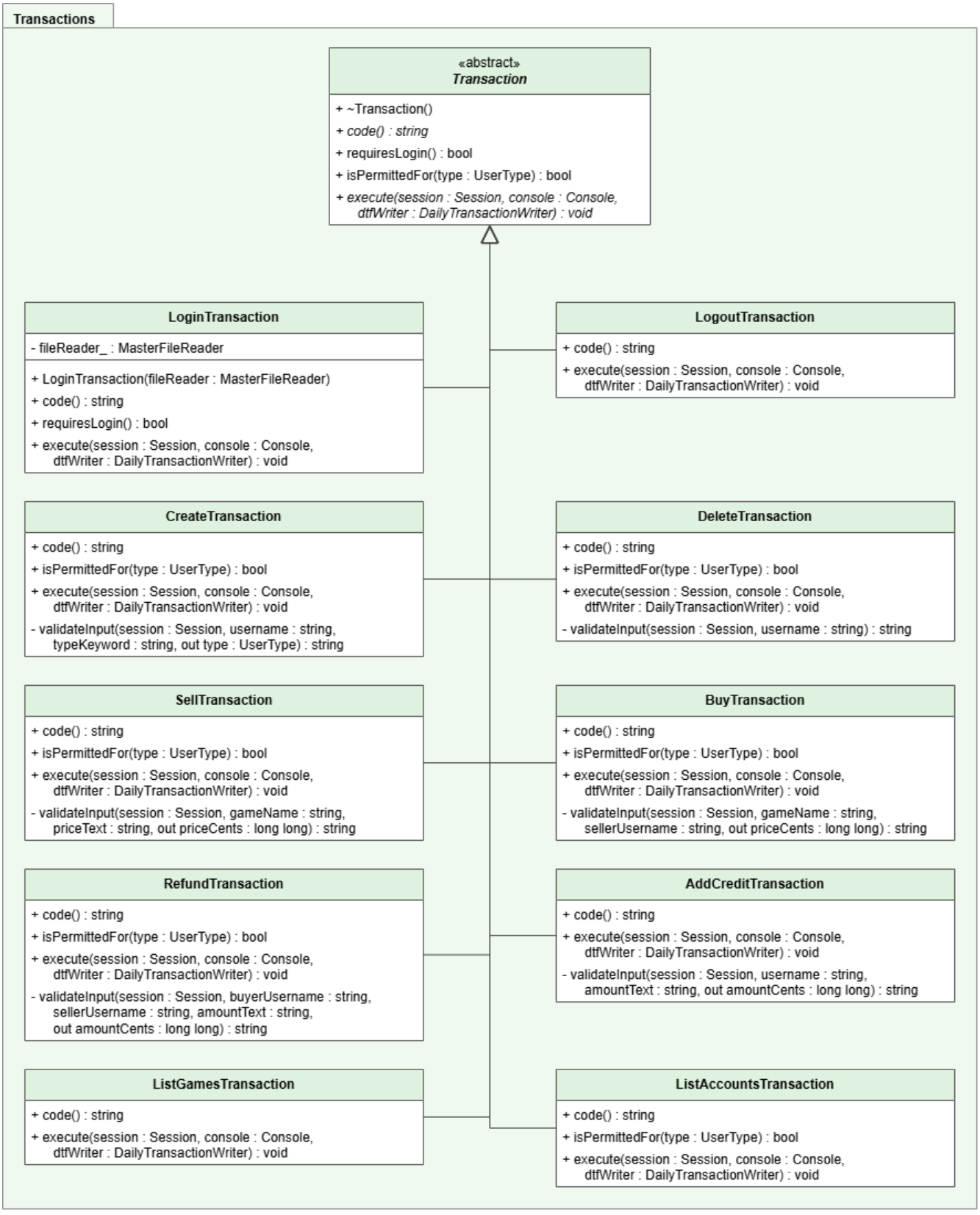


Figure 5. *Detailed classes: the abstract Transaction and its ten subclasses.*

5. Class and method intention table

Table 4 lists every class (shaded rows) and every method, grouped by package, using exactly the names and signatures in Figures 3 to 5. Each class row also gives the kind of class and its source files. Signatures use the same notation as the diagrams: + public, - private, underlined = static, italic = abstract. Records (structs) have attributes only; their fields are shown in Figure 3.

Table 4. Intention of every class and method

Class / method (as in the diagrams)	Intention (one sentence)
Package: Program entry	
main main program; main.cpp	Program entry point: checks the command-line arguments, then creates and runs the Front End.
`+ main(argc : int, argv : char*[]) : int`	Checks that four arguments were given and that the three master files can be opened, then creates the FrontEnd, runs it and returns its exit status.
Package: Control	
FrontEnd class; FrontEnd.h / FrontEnd.cpp	Runs the Front End: reads one transaction code per line, rejects codes that are unknown, out of order or not permitted for the logged-in account, and passes accepted codes to the matching Transaction.
`+ FrontEnd(input : istream, output : ostream, fileReader : MasterFileReader, dtfFilePrefix : string)`	Connects the console to the given streams, copies the file reader, passes the DTF file prefix to the DailyTransactionWriter, and registers the ten transactions.
`+ run() : int`	Reads transaction codes until the input ends, processing each one, and returns 0.
`- processTransactionCode(code : string) : void`	Finds the transaction for a code and either executes it or prints why it was rejected.
`- rejectionReason(transaction : Transaction) : string`	Returns why the transaction cannot run now (not logged in, already logged in, or not permitted for this account type), or an empty string if it can.
`- registerTransaction(transaction : unique_ptr<Transaction>) : void`	Adds a transaction to the lookup table under its own transaction code.
Package: Transactions	
Transaction abstract class; transactions/Transaction.h / Transaction.cpp	Abstract base class defining what every transaction provides (its code word, whether it needs a logged-in user, which account types may use it, and how it runs), so FrontEnd can treat them alike.
`+ ~Transaction()`	Lets FrontEnd delete any transaction safely through a Transaction pointer.
`+ code() : string`	Returns the word that selects this transaction, such as "buy".
`+ requiresLogin() : bool`	Returns true if the transaction is accepted only while a user is logged in (true for every transaction except login).
`+ isPermittedFor(type : UserType) : bool`	Returns true if an account of the given type may use this transaction (true unless a subclass restricts it).
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Asks for the transaction's fields, validates them, and on success updates the session, records a DTF line and confirms; otherwise prints the reason and changes nothing.
LoginTransaction class, subclass of Transaction; transactions/LoginTransaction.h / .cpp	Starts a session: reads the user accounts file, asks for a username, and if it belongs to an account reads the available games and game collection files and starts accepting transactions.

Class / method (as in the diagrams)	Intention (one sentence)
`+ LoginTransaction(fileReader : MasterFileReader)`	Keeps a reference to the reader that loads the master files at each login.
`+ code() : string`	Returns "login", the word that selects this transaction.
`+ requiresLogin() : bool`	Returns false, because login is the only transaction accepted while no one is logged in.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Reads the accounts file, prompts for a username, and starts the session with fresh copies of all three master files if the username exists; otherwise rejects it.
LogoutTransaction class, subclass of Transaction; transactions/LogoutTransaction.h / .cpp	Ends the session: writes the session's Daily Transaction File and returns the Front End to accepting only login.
`+ code() : string`	Returns "logout", the word that selects this transaction.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Writes the session's DTF records and the 00 record to a new file, ends the session and confirms the logout.
CreateTransaction class, subclass of Transaction; transactions/CreateTransaction.h / .cpp	Admin-only transaction that creates a new account from a username and an account type; the new account starts with 0.00 credit.
`+ code() : string`	Returns "create", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns true only for admin accounts.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for the new username and the account type, then adds the account to the session and records a 01 line if both are valid.
`- validateInput(session : Session, username : string, typeKeyword : string, out type : UserType) : string`	Checks the username's length and uniqueness and converts the account-type word to a UserType; returns an error message, or an empty string if both fields are valid.
DeleteTransaction class, subclass of Transaction; transactions/DeleteTransaction.h / .cpp	Admin-only transaction that removes an existing account (never the logged-in user's) and cancels that user's games for sale for the rest of the session.
`+ code() : string`	Returns "delete", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns true only for admin accounts.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for a username, then removes the account and its games for sale from the session and records a 02 line if the deletion is allowed.
`- validateInput(session : Session, username : string) : string`	Checks that the username belongs to an existing account other than the logged-in user's; returns an error message, or an empty string if the deletion is allowed.
SellTransaction class, subclass of Transaction; transactions/SellTransaction.h / .cpp	Lists a game for sale by the logged-in user (any account type except buy-standard); the game cannot be bought until the next session.
`+ code() : string`	Returns "sell", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns false for buy-standard accounts and true for all others.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for the game name and the price, then reserves the name for this session and records a 03 line if both are valid.
`- validateInput(session : Session, gameName : string, priceText : string, out priceCents : long long) : string`	Checks the game name's length and uniqueness and converts the price to cents (0.00 to 999.99); returns an error message, or an empty string if the sale is valid.
BuyTransaction class, subclass of Transaction; transactions/BuyTransaction.h / .cpp	Buys a game that is for sale (any account type except sell-standard): moves the price from the buyer to the seller and adds the game to the buyer's collection.
`+ code() : string`	Returns "buy", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns false for sell-standard accounts and true for all others.

Class / method (as in the diagrams)	Intention (one sentence)
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for the game name and the seller's username, then transfers the price, adds the game to the buyer's collection and records a 04 line if the purchase is valid.
`- validateInput(session : Session, gameName : string, sellerUsername : string, out priceCents : long long) : string`	Checks that the game is for sale by that seller (who still has an account) and that the buyer is not the seller, does not already own it and has enough credit, and gives back the price; returns an error message, or an empty string if the purchase is valid.
RefundTransaction class, subclass of Transaction; transactions/RefundTransaction.h / .cpp	Admin-only transaction that moves an amount of credit from a seller's account to a buyer's account.
`+ code() : string`	Returns "refund", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns true only for admin accounts.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for the buyer's username, the seller's username and the amount, then moves the amount from seller to buyer and records a 05 line if the refund is valid.
`- validateInput(session : Session, buyerUsername : string, sellerUsername : string, amountText : string, out amountCents : long long) : string`	Checks that both users exist and converts the amount to cents, which must be above 0 and no more than the seller's credit; returns an error message, or an empty string if the refund is valid.
AddCreditTransaction class, subclass of Transaction; transactions/AddCreditTransaction.h / .cpp	Adds credit to an account, at most 1000.00 per account per session: standard users add to their own account, while an admin names the account to credit.
`+ code() : string`	Returns "addcredit", the word that selects this transaction.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prompts for the amount (and, for an admin, then the username), then adds the credit and records a 06 line if the request is valid.
`- validateInput(session : Session, username : string, amountText : string, out amountCents : long long) : string`	Checks that the account exists and converts the amount to cents, which must be above 0, keep this session's total for the account within 1000.00 and keep its balance within the maximum; returns an error message, or an empty string if the request is valid.
ListGamesTransaction class, subclass of Transaction; transactions/ListGamesTransaction.h / .cpp	Displays every game available for purchase in this session with its seller and price; it writes no DTF record.
`+ code() : string`	Returns "list_games", the word that selects this transaction.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prints one line per game available for purchase (name, seller, price), or a message saying that no games are available.
ListAccountsTransaction class, subclass of Transaction; transactions/ListAccountsTransaction.h / .cpp	Admin-only transaction from the client's Week 4 update that displays every active account in this session with its account type and credit; it writes no DTF record.
`+ code() : string`	Returns "list_accounts", the word that selects this transaction.
`+ isPermittedFor(type : UserType) : bool`	Returns true only for admin accounts.
`+ execute(session : Session, console : Console, dtfWriter : DailyTransactionWriter) : void`	Prints one line per active account (username, account type code and credit) in username order, including accounts created this session and leaving out accounts deleted this session.
Package: Session state	
Session class; Session.h / Session.cpp	Holds everything known during one login: the logged-in user, in-memory copies of the three master files that change as transactions succeed, and this session's records of games listed and credit added.
`+ begin(username : string, accounts : map<string, UserAccount>, listings : map<string, GameListing>, collection : vector<OwnedGame>) : void`	Starts a session for the given user with freshly read master-file data and clears all per-session records.
`+ end() : void`	Ends the session so that only login is accepted next.

Class / method (as in the diagrams)	Intention (one sentence)
`+ isLoggedIn() : bool`	Returns true while a user is logged in.
`+ currentUser() : UserAccount`	Returns the logged-in user's account.
`+ findAccount(username : string) : UserAccount*`	Returns the account with this username, or nullptr if there is none (including accounts deleted this session).
`+ accounts() : map<string, UserAccount>`	Returns every active account in this session, in username order, as a read-only view for list_accounts.
`+ addAccount(account : UserAccount) : void`	Adds an account created this session so that later transactions can use it.
`+ removeAccount(username : string) : void`	Removes a deleted account and cancels every game that user has for sale, for the rest of the session.
`+ addCredit(username : string, amountCents : long long) : void`	Increases an account's credit and adds the amount to that account's total of credit added this session.
`+ transferCredit(fromUsername : string, toUsername : string, amountCents : long long) : void`	Moves credit from one account to another (used by buy and refund).
`+ creditAddedThisSession(username : string) : long long`	Returns the total credit added to an account by addcredit in this session, in cents.
`+ findListing(gameName : string) : GameListing*`	Returns the game for sale under this name, or nullptr if there is none; games listed this session are never returned, so they cannot be bought until the next session.
`+ listings() : map<string, GameListing>`	Returns every game available for purchase in this session, in name order.
`+ isGameNameTaken(gameName : string) : bool`	Returns true if a game with this name is already for sale or was listed earlier this session.
`+ reserveGameName(gameName : string) : void`	Records a game listed this session so its name stays unique while it stays unavailable for purchase.
`+ ownsGame(username : string, gameName : string) : bool`	Returns true if the user's game collection already contains this game.
`+ addOwnedGame(username : string, gameName : string) : void`	Adds a purchased game to the buyer's collection for the rest of the session.
UserAccount struct; UserAccount.h	One line of the current user accounts file held in memory: username, account type and credit.
`username : string, type : UserType, creditCents : long long = 0`	The fields of the record; it has no methods.
GameListing struct; GameListing.h	One line of the available games file held in memory: game name, seller's username and price.
`gameName : string, sellerUsername : string, priceCents : long long = 0`	The fields of the record; it has no methods.
OwnedGame struct; OwnedGame.h	One line of the game collection file held in memory: a game and the username of the account that owns it.
`gameName : string, ownerUsername : string`	The fields of the record; it has no methods.
UserType enum; UserType.h	The four account types: Admin (AA), FullStandard (FS), BuyStandard (BS) and SellStandard (SS).
`Admin, FullStandard, BuyStandard, SellStandard`	The values of the enum; it has no methods.
Package: Input / Output	
Console class; Console.h / Console.cpp	Wraps standard input and output: reads one line at a time with any trailing carriage return removed, shows prompts and prints messages.
`+ Console(input : istream, output : ostream)`	Connects the console to the given input and output streams (std::cin and std::cout when the program runs).

Class / method (as in the diagrams)	Intention (one sentence)
`+ readLine(out line : string) : bool`	Reads the next input line, removing a trailing carriage return left by Windows line endings; returns false when the input has ended.
`+ prompt(question : string, out answer : string) : bool`	Prints a question on its own line and reads the answer; returns false when the input has ended.
`+ showMessage(message : string) : void`	Prints a message on its own line.
MasterFileReader class; MasterFileReader.h / MasterFileReader.cpp	Reads the three master files (current user accounts, available games and game collection) into in-memory records, stopping at each file's END line.
`+ MasterFileReader(userAccountsPath : string, availableGamesPath : string, gameCollectionPath : string)`	Keeps the paths of the three master files given on the command line.
`+ canOpenAllFiles() : bool`	Returns true only if all three master files can be opened, so a wrong path is reported at start-up.
`+ readUserAccounts() : map<string, UserAccount>`	Reads each 28-character account line (username, type, credit) into a map keyed by username.
`+ readAvailableGames() : map<string, GameListing>`	Reads each 49-character line (game name, seller, price) into a map keyed by game name.
`+ readGameCollection() : vector<OwnedGame>`	Reads each 42-character line (game name, owner) into a list.
`- readRecordLines(path : string, keyFieldWidth : size_t) : vector<string>`	Returns the lines of a master file that come before its END line, with trailing carriage returns removed.
DailyTransactionWriter class; DailyTransactionWriter.h / .cpp	Formats this session's successful transactions as fixed-width Daily Transaction File records, keeps them in memory, and writes them with the 00 end-of-session record to a new file at logout.
`+ DailyTransactionWriter(filePrefix : string)`	Keeps the prefix used to name each session's file (session N is written to <prefix>_session<N>.txt).
`+ recordCreate(username : string, type : UserType) : void`	Adds a 01 record for a new account: username, account type code and credit 000000.00.
`+ recordDelete(username : string) : void`	Adds a 02 record for a deleted account: username, blank type and credit 000000.00.
`+ recordSell(gameName : string, sellerUsername : string, priceCents : long long) : void`	Adds a 03 record: game name, seller's username and price.
`+ recordBuy(gameName : string, sellerUsername : string, buyerUsername : string, priceCents : long long) : void`	Adds a 04 record: game name, seller's username, buyer's username and price.
`+ recordRefund(buyerUsername : string, sellerUsername : string, amountCents : long long) : void`	Adds a 05 record: buyer's username, seller's username and refund amount.
`+ recordAddCredit(username : string, amountCents : long long) : void`	Adds a 06 record: username, blank type and the amount of credit added.
`+ writeSessionFile() : bool`	Adds the 00 record, writes this session's records to the next session file and clears them; returns false if the file could not be written.
`- addAccountRecord(transactionCode : string, username : string, typeCode : string, creditCents : long long) : void`	Formats and stores one record in the XX UUUUUUUUUUUUUUU TT CCCCCCCCC layout shared by the 01, 02, 06 and 00 records.
`- makeEndRecord() : string`	Builds the 00 end-of-session record that closes every Daily Transaction File.
Package: Utilities	
UserTypeCodes utility class (static methods only); UserTypeCodes.h / UserTypeCodes.cpp	Converts account types to and from the two-letter file codes (AA, FS, BS, SS) and the words typed at the create prompt (admin, full-standard, buy-standard, sell-standard).

Class / method (as in the diagrams)	Intention (one sentence)
+ toFileCode(type : UserType) : string	Returns the two-letter code for an account type, such as FS for FullStandard.
+ fromFileCode(code : string, out type : UserType) : bool	Converts a two-letter code read from the accounts file; returns false if the code is not recognised.
+ fromKeyword(keyword : string, out type : UserType) : bool	Converts a typed account-type word such as full-standard; returns false if the word is not recognised.
Money utility class (static methods only); Money.h / Money.cpp	Converts dollar amounts between text and whole cents, so that every comparison with a limit such as 999.99 or 1000.00 is exact.
+ parse(text : string, out cents : long long) : bool	Converts text such as 15, 15.5 or 015.00 to cents; returns false for blank, negative or non-numeric text, or more than two decimal places.
+ toFileField(cents : long long, width : size_t) : string	Formats cents as a zero-filled field with two decimals; for example 2000 cents in 9 characters becomes 000020.00.
+ toDisplay(cents : long long) : string	Formats cents for console messages; for example 2000 becomes 20.00.
FixedWidthFormat utility class (static methods only); FixedWidthFormat.h / FixedWidthFormat.cpp	Defines the width of every fixed-width field and pads and extracts those fields, so the master files, the Daily Transaction File, list_games and list_accounts share one definition.
USERNAME_WIDTH = 15, TYPE_WIDTH = 2, CREDIT_WIDTH = 9, GAME_NAME_WIDTH = 26, PRICE_WIDTH = 6	Field widths (static constants) used by every fixed-width reader and writer.
+ padRight(text : string, width : size_t) : string	Left-justifies text in a field of the given width, filling with spaces; every caller passes text that already fits the width (checked by InputRules, or read from a field of that width).
+ extractField(line : string, start : size_t, width : size_t) : string	Returns the field starting at the given position with trailing spaces removed (empty if the line is too short).
InputRules utility class (static methods only); InputRules.h / InputRules.cpp	Holds the limits the Front End enforces and the checks for typed usernames and game names, so each limit is defined once.
MAX_USERNAME_LENGTH = FixedWidthFormat::USERNAME_WIDTH, MAX_GAME_NAME_LENGTH = FixedWidthFormat::GAME_NAME_WIDTH, MAX_PRICE_CENTS = 99999, MAX_CREDIT_ADDED_PER_SESSION_CENTS = 100000, MAX_BALANCE_CENTS = 99999999	Named limits (static constants) used by the validation checks.
+ isValidUsername(username : string) : bool	Returns true if a username is 1 to 15 characters long and does not start or end with a space.
+ isValidGameName(gameName : string) : bool	Returns true if a game name is 1 to 26 characters long and does not start or end with a space.

6. Run-time behaviour

6.1 Transaction gates

`FrontEnd::rejectionReason` decides whether a known transaction code may run, before any prompt is shown. Table 5 lists every case.

Table 5. What happens to each line read as a transaction code

Situation	Result
The line is not one of the ten codes (including a blank line)	`Error: "<code>" is not a valid transaction code.` Nothing else is read for it.
Nobody is logged in and the code is not login	`Error: please log in before using "<code>".` No prompt is shown.
Someone is logged in and the code is login	`Error: already logged in. Log out before logging in again.` No prompt is shown.
The logged-in account type may not use the transaction (Table 3)	`Error: your account type cannot use "<code>".` No prompt is shown.
Otherwise	The transaction's `execute` runs: it prompts for all of its fields, validates them, then either applies the transaction or prints the reason it was refused.

6.2 Session lifecycle

- **Login** reads the accounts file, prompts for the username and, if it is valid, reads the available games and game collection files, then calls `Session::begin`. Each login starts from the files as they are on disk.
- **During the session**, successful transactions change only the in-memory `Session` and add records to the `DailyTransactionWriter`. A game listed by sell is reserved by name but is not for sale until a later session; deleting an account also cancels its games for sale.
- **Logout** writes the session's records and the 00 record to `<dtf prefix>_session<N>.txt`, then calls `Session::end`; only login is accepted afterwards. Several sessions may follow one another in a single run.
- **End of input** ends the program with exit status 0. If a session is still open, its records are discarded, because only logout writes a DTF.

6.3 One transaction from start to finish

Figure 6 follows one accepted buy and the logout that writes its record. The `alt` box shows the two outcomes of validation: an invalid buy changes nothing and records nothing; a valid buy moves the price from buyer to seller, adds the game to the buyer's collection, and adds a 04 record that is written to the DTF at logout.

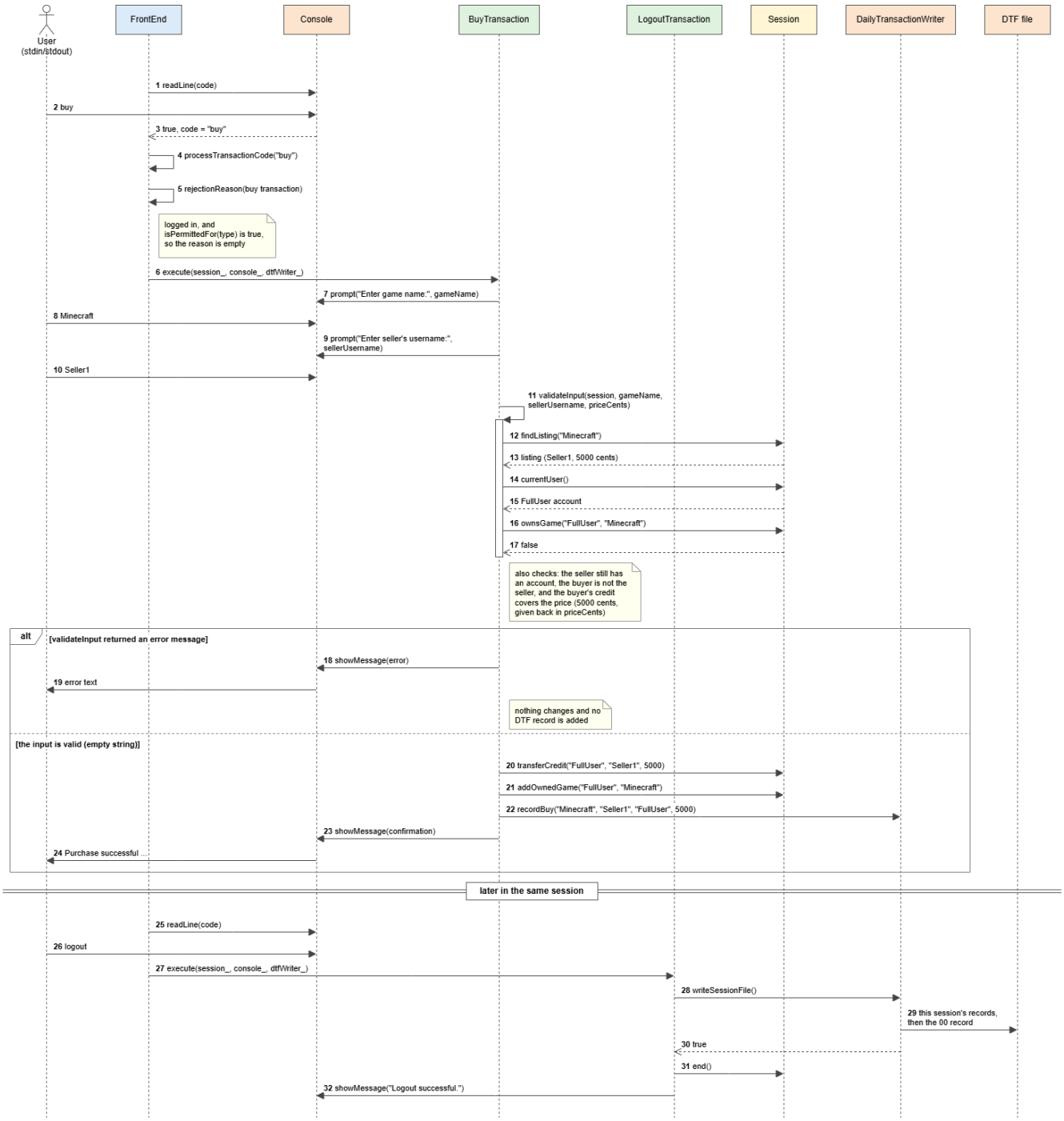


Figure 6. *Sequence diagram: an accepted buy (FullUser buys Minecraft from Seller1 for 50.00), then logout.*

6.4 Order of validation checks

Each `validateInput` applies its checks in a fixed order and reports only the first one that fails, so the same input always produces the same message. Table 6 gives the order.

Table 6. Validation checks, in the order they are applied

Transaction	Checks
login	1. The username belongs to an account in the freshly read accounts file (a blank username never does).
create	1. The username is 1 to 15 characters and does not start or end with a space. 2. No account already has that username. 3. The type word is admin, full-standard, buy-standard or sell-standard.
delete	1. The username belongs to an existing account. 2. It is not the logged-in user's account.
sell	1. The game name is 1 to 26 characters (client clarification 1; the written specification says 25) and does not start or end with a space. 2. No game with that name is for sale or was listed earlier this session. 3. The price is a valid amount from 0.00 to 999.99.
buy	1. The game is for sale by that seller, and the seller still has an account (games listed this session, and games of accounts deleted this session, are not for sale). 2. The buyer is not the seller. 3. The buyer does not already own the game. 4. The buyer's credit is at least the price.
refund	1. The buyer's username belongs to an account. 2. So does the seller's. 3. The amount is a valid amount. 4. It is greater than 0.00. 5. It is no more than the seller's credit.
addcredit	1. The amount is a valid amount. 2. It is greater than 0.00. 3. (Admin) the username belongs to an account. 4. The credit added to that account this session, plus this amount, is at most 1000.00. 5. The new balance is at most 999,999.99.

7. Daily Transaction File records

`DailyTransactionWriter` writes one record per successful transaction, in the order they happened, and ends every session with the 00 record. Alphabetic fields are left-justified and padded with spaces; money fields are zero-filled with two decimals. Table 7 gives each layout (␣ is one space). These layouts match the expected DTF files of our Phase 1 tests.

Table 7. Record layouts

Code	Transaction	Written by	Layout	Length
01	create	`recordCreate`	01␣username(15)␣type(2)␣000000.00	31
02	delete	`recordDelete`	02␣username(15)␣␣␣␣000000.00 (type left blank)	31
03	sell	`recordSell`	03␣game(26)␣seller(15)␣price(6)	52
04	buy	`recordBuy`	04␣game(26)␣seller(15)␣buyer(15)␣price(6)	68
05	refund	`recordRefund`	05␣buyer(15)␣seller(15)␣amount(9)	44
06	addcredit	`recordAddCredit`	06␣username(15)␣␣␣␣amount(9) (type left blank; the amount added)	31
00	end of session	`writeSessionFile`	00, then 20 spaces, then 000000.00	31

Example records (from our Phase 1 expected DTF files; the ruler counts columns):

```
         1         2         3         4         5         6
123456789012345678901234567890123456789012345678901234567890123456789012345678
01 NewAdmin        AA 000000.00
02 David              000000.00
03 Portal                    AdminUser       020.00
```

```
04 Minecraft                  Seller1         FullUser        050.00
05 Buyer1          Seller1         000020.00
06 FullUser           000100.00
00                    000000.00
```

- Records 01, 02, 06 and 00 share one layout and are all built by the private `addAccountRecord` method, so that layout is written once.
- Each line ends with a newline written in text mode, which is a CRLF on Windows (the line ending used by our test files). The Phase 3 test scripts compare files ignoring line-ending differences.
- A failed or incomplete transaction writes no record (client clarification 7); list_games and list_accounts write none either.

8. Requirements coverage and Phase 2 scope

8.1 Where each requirement is handled

Table 8 maps every requirement to the classes and methods that implement it, and to the Phase 1 tests that will check it in Phase 3.

Table 8. Requirements traceability

Requirement	Design element	Phase 1 tests
Read the accounts file before login; ask for the username; read the games (and collection) files after it is accepted	`LoginTransaction::execute`, `MasterFileReader::readUserAccounts`, `readAvailableGames`, `readGameCollection`	login_01 to login_05, login_09
Only login before a login; no second login before a logout	`FrontEnd::rejectionReason`, `Transaction::requiresLogin`, `LoginTransaction::requiresLogin`	login_06 to login_08, logout_02 to logout_10, list_games_07, list_games_08
Privileged and semi-privileged transactions refused for other account types	`isPermittedFor` in `CreateTransaction`, `DeleteTransaction`, `SellTransaction`, `BuyTransaction`, `RefundTransaction`, `ListAccountsTransaction`	create_05, delete_02, sell_04, buy_04, refund_02 (list_accounts: none yet)
Logout writes the DTF; then only login is accepted	`LogoutTransaction::execute`, `DailyTransactionWriter::writeSessionFile`, `Session::end`	logout_01, transaction_output_11
create: name at most 15 characters and unique; account type; 0.00 credit; 01 record	`CreateTransaction::validateInput`, `InputRules::isValidUsername`, `UserTypeCodes::fromKeyword`, `Session::addAccount`, `DailyTransactionWriter::recordCreate`	create_01 to create_11
delete: existing user, not the current user; their games for sale cancelled; 02 record	`DeleteTransaction::validateInput`, `Session::removeAccount`, `DailyTransactionWriter::recordDelete`	delete_01 to delete_06 (delete_07 checks the collection: Back End)
sell: not buy-standard; price at most 999.99; name at most 26 characters and unique; not for sale until a later session; 03 record	`SellTransaction::validateInput`, `Session::isGameNameTaken`, `Session::reserveGameName`, `DailyTransactionWriter::recordSell`	sell_01 to sell_14 (sell_15: later sessions re-read the unchanged master files)
buy: not sell-standard; game for sale by that seller; enough credit; not already owned; not the buyer's own game; price moved; game added; 04 record	`BuyTransaction::validateInput`, `Session::findListing`, `ownsGame`, `transferCredit`, `addOwnedGame`, `DailyTransactionWriter::recordBuy`	buy_01 to buy_12 (buy_13 logs in as a sell-standard user, so it stops at the permission check, not the own-game rule)
refund: admin only; both users exist; amount above 0.00 and no more than the seller's credit; credit moved; 05 record	`RefundTransaction::validateInput`, `Session::transferCredit`, `DailyTransactionWriter::recordRefund`	refund_01 to refund_13 (refund_14 checks the collection, a Back End effect; refund_15: later sessions re-read the unchanged master files)
addcredit: amount (admin: then an existing username); at most 1000.00 per account per session; balance at most the maximum; 06 record	`AddCreditTransaction::execute`, `validateInput`, `Session::addCredit`, `Session::creditAddedThisSession`, `DailyTransactionWriter::recordAddCredit`	addcredit_01 to addcredit_11
list_games: logged-in users of every type; shows game, seller and price; no DTF record	`ListGamesTransaction::execute`, `Session::listings`	list_games_01 to list_games_12
list_accounts (Week 4 update): admin only; shows every active account with its type and credit; no DTF record	`ListAccountsTransaction::isPermittedFor`, `ListAccountsTransaction::execute`, `Session::accounts`	None yet: the update came after Phase 1, so Phase 3 adds its tests

Requirement	Design element	Phase 1 tests
Never crash; handle bad input of every kind	`Console::readLine` (end of input), `Money::parse` (never throws), every `validateInput`, `FrontEnd::processTransactionCode`	invalid_input_01 to invalid_input_10
DTF format: field widths, padding, zero fill, two decimals, 00 record at the end	`FixedWidthFormat::padRight`, `Money::toFileField`, `DailyTransactionWriter::addAccountRecord`, `writeSessionFile`	transaction_output_01 to transaction_output_18

8.2 What the Phase 2 first version includes

- Every class and method in this document, with the names and signatures shown in Figures 3 to 5 and Table 4. Enough of each transaction is implemented to run on some sample input; some methods are still partial and will be completed and debugged in Phase 3
- A comment at the start of every class and method stating its interface and intention, and a comment at the start of `main.cpp` stating the program's intention, its input and output files, and how to run it.
- A clean build with `g++ -std=c++17 -Wall -Wextra`, run by hand on a few sample sessions.
- Not yet included: running the Phase 1 tests, comparing outputs automatically, and fixing the failures they find. That is Phase 3.
- list_accounts arrived after Phase 1, so none of the tests cover it; Phase 3 adds test cases for it (Table 8).